

Introduction to Java Swing and Event-Driven Programming

1 Event-Driven Programming

In traditional programming (which you are familiar with from procedural and object-oriented programming), execution follows a **top-down control flow**:

`main()` → `method calls` → `program terminates`

However, graphical user interfaces (GUIs) operate under a different paradigm known as:

Event-Driven Programming

Key Idea

Instead of the program controlling execution, the **user controls the flow** through events such as:

- Clicking a button
- Typing on the keyboard
- Moving the mouse

The program **waits** for events and responds accordingly.

Conceptual Model

- You define **components** (buttons, windows, text fields)
- You attach **listeners** (event handlers)
- The system triggers your code when an event occurs

2 What is Java Swing?

Java Swing is a **GUI toolkit** that allows us to build graphical applications in Java.

Core Components

- `JFrame` — the main window
- `JLabel` — displays text
- `JButton` — clickable button
- `JPanel` — container for grouping components

How Swing Works

1. Create a window (`JFrame`)
2. Add components (buttons, labels, etc.)
3. Attach event listeners
4. Make the window visible

3 Example 1: Hello World (Swing Version)

```
import javax.swing.*;

public class HelloWorldGUI {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Hello World");

        JLabel label = new JLabel("Hello, World!");
        frame.add(label);

        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Discussion:

- A window is created using `JFrame`
- A label is added to display text
- The program remains open, waiting for user interaction

4 Example 2: Adding a Button (No Behavior)

```
import javax.swing.*;

public class ButtonExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Button Example");

        JButton button = new JButton("Click Me");

        frame.add(button);

        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Observation:

- The button exists but does nothing
- No event handler is attached yet

5 Example 3: Button with Action

```
import javax.swing.*;
import java.awt.event.*;

public class ButtonActionExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Button Action");

        JButton button = new JButton("Click Me");

        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                System.out.println("Button was clicked!");
            }
        });

        frame.add(button);

        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

```
}
```

Key Idea:

- We attach an `ActionListener`
- When the button is clicked, the method runs

6 Example 4: Persistent State (Counter)

```
import javax.swing.*;
import java.awt.event.*;

public class CounterExample {

    static int count = 0;

    public static void main(String[] args) {
        JFrame frame = new JFrame("Counter Example");

        JButton button = new JButton("Increment");
        JLabel label = new JLabel("Count: 0");

        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                count++;
                label.setText("Count: " + count);
            }
        });

        frame.setLayout(new java.awt.FlowLayout());
        frame.add(button);
        frame.add(label);

        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Important Concept:

- The variable `count` persists in memory
- Each click modifies program state
- GUI updates reflect state changes

7 Summary

- Swing programs are **event-driven**, not sequential
- Components generate events
- Listeners define how the program responds
- State can persist and evolve through user interaction

8 Example 5: Layout Management with BorderLayout

So far, we have placed components into a frame without explicitly controlling their positions. Java Swing uses **layout managers** to control how components are arranged.

BorderLayout

One of the most important layout managers is:

BorderLayout

It divides the frame into five regions:

- **North** (top)
- **South** (bottom)
- **East** (right)
- **West** (left)
- **Center** (middle)

Key Idea

Each region can hold one component, and the layout manager automatically resizes them when the window changes size.

Example Code

```
import javax.swing.*;
import java.awt.*;

public class BorderLayoutExample {
    public static void main(String[] args) {

        JFrame frame = new JFrame("BorderLayout Example");
```

```
// Set size of the window
frame.setSize(500, 400);

// Explicitly set layout manager
frame.setLayout(new BorderLayout());

// Create components
JButton northButton = new JButton("North");
JButton southButton = new JButton("South");
JButton eastButton = new JButton("East");
JButton westButton = new JButton("West");
JButton centerButton = new JButton("Center");

// Add components to specific regions
frame.add(northButton, BorderLayout.NORTH);
frame.add(southButton, BorderLayout.SOUTH);
frame.add(eastButton, BorderLayout.EAST);
frame.add(westButton, BorderLayout.WEST);
frame.add(centerButton, BorderLayout.CENTER);

frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}
```

Discussion

- The frame size is explicitly controlled using:

```
frame.setSize(width, height)
```

- The layout manager determines how components are positioned:

```
frame.setLayout(new BorderLayout())
```

- Each component is assigned a **region**:

- Top → `BorderLayout.NORTH`
- Bottom → `BorderLayout.SOUTH`
- Left → `BorderLayout.WEST`
- Right → `BorderLayout.EAST`
- Middle → `BorderLayout.CENTER`

- The **Center region expands** to fill remaining space.

Important Insight

Unlike manually setting coordinates, layout managers:

- Automatically adjust component sizes
- Respond to window resizing
- Enforce consistent structure across platforms

This is the preferred way to design professional GUIs in Java.